

ECE 3405: Image Segmentation & Recognition of Transparent Objects for Drone Application

By Samuel Kim and Rafael Mirzoyan

Introduction

In recent years, drones have become increasingly important across a wide range of fields—from military reconnaissance and offensive operations, to terrain surveillance and commercial delivery. While many drones are still manually operated, advancements in artificial intelligence(AI) have led to the development of AI-assisted navigation systems and fully autonomous drones. These rely on models trained for situational awareness and pattern recognition, enabling drones to interpret their surroundings and make informed decisions in real-time.

At the core of this technology lies object detection, which integrates sensor data with neural networks to identify and analyze objects for navigation, safety and data collection. However, transparent and reflective surfaces remain a significant challenge. Unlike opaque objects, these materials do not have consistent color or texture and instead reflect or transmit parts of the surrounding environment. This often confuses models such as convolutional neural networks (CNNs), which struggle to distinguish such objects as separate entities.

While many objects can be mapped using conventional parameters and datasets, transparent objects have long posed a challenge due to their challenge of conventional assumptions the neural networks are normally built on. Transparent objects have neither a uniform texture or color, displaying contextual parts of the environment either behind or around them. This makes it difficult for networks such as CNN to recognize them as uniform and individual objects to classify. Across all applications, drones use these networks for vision-based obstacle avoidance, relying on trained models to detect buildings, trees, even people and animals to plot flight paths. However, transparent objects often trick the neural network into believing there is open space by reflecting or displaying surrounding objects/people,

or failing to detect glass objects due to their lack of texture and color. Even with the use of additional sensors, these models have a hard time navigating such objects due to refraction, depth perception, and unpredictable reflection, which is why research has been done on how to optimally train neural networks to recognize transparent and reflective objects purely through visual inspection.

Related Works

The paper *Segmenting Transparent Objects in the Wild* introduces a new framework designed to improve the segmentation of transparent objects by focusing on label-agnostic learning. That is, instead of relying on predefined class labels, the model focuses on recognizing transparency itself as a distinct visual property. While this paper leverages both RGB and depth information to better capture object boundaries and reduce segmentation errors, we found that the collected RGB data and corresponding masks alone can provide valuable insight and serve as a strong foundation for training our own neural network. We also consulted other papers such as *Object detection characteristics in a learning factory environment using YOLOv8* and *SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation* for more model and architecture specific questions and ideas.

Background

Drones must make snap decisions in response to new elements and obstacles that enter their field of vision, requiring real-time parsing of images. As a result, we decided to utilize a Convolutional Neural Network. CNNs use a variety of filters to form features by processing the image locally, which allows for generalization. This is extremely important for drones because of their constantly changing perspectives in flight. CNNs are also lightweight and run real-time inference, minimizing computing requirements for a tight information pipeline. Other options such as vision transformers, which have many more parameters, make it much more difficult for drones to make real-time software decisions.

Transparent objects have defining features that are much more subtle, based on global data. These can include distortions of the background, discontinuities in depth, and refraction patterns.

Encoder-decoder architecture CNNs excel at this because of their pixel-level feature detection. The

encoder compresses the image into abstract, high-level features such as shapes and textures by parsing the image layer by layer. From there, a highly compressed feature-focused orientation of the original image is sent to the decoder, which attempts to identify object centers for the bounding box and pixel class for segmentation and classification. We decided to utilize YOLOv8, an open source CNN built for bounding-boxes and image segmentation. YOLOv8 specifically utilizes an anchor-free approach for model simplification, contributing to faster training and inference. A more detailed diagram of the whole process can be seen below in figure 1.

Figure 1: Schematic representation of Yolov8

YOLO utilizes 53 convolutional layers, as well as c2f modules that include context information of the image in the recognition process to potentially improve prediction accuracy. This is the key point that led us to choosing YOLO, as transparent objects rely on contextual cues for pattern recognition.

Our approach was broken down into 2 steps. First, we train our YOLO model to recognize transparency as a separate class. From there, we run it in an ensemble with a separate YOLO model designed to recognize commonplace objects. The trans10K dataset has 10,000 images of transparent

objects with their associated pixel masks, which would be used as ground truths for the model to compare against. However, YOLO cannot take raw masks as data to read, so it is necessary to convert the masks into usable text files first.

After finding the root path of our files, our code would read and preprocess each mask, converting to gray scale if 3-channel, as seen below in figure 2.

```
#Reads pixel mask
mask = cv2.imread(mask_path, cv2.IMREAD_UNCHANGED)
if mask is None:
    print(f"[WARN] Could not read mask {mask_path}") #Error message for pixel mask
    continue

#Convert to gray-scale
if mask.ndim == 3:
    gray = cv2.cvtColor(mask, cv2.COLOR_BGR2GRAY)
else:
    gray = mask
```

Figure 2: Sample code for mask preprocessing

This feature was added as a fail-safe, as well as making the code usable for other masks for other datasets. Afterwards, the code reads contours from the converted binary mask, skipping if none are found as seen below in figure 3.

```
#Creates binary mask because background, 255 = object
binary = (gray > 0).astype(np.uint8) * 255

# Find contours of the object(s)
contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
if not contours:
    print(f"[WARN] No contours found in mask {mask_name}")
    continue
```

Figure 3: Sample code for contour reading

Next, we find the size of the image using the built-in function from cv2, extracting the height and width from the matrix. Afterwards, we filter contours, compute bounding boxes, and normalize the contours for YOLO to read, outputting them into a file readable for training, seen below in figure 4.

```
with open(label_path, "w") as f:
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < 30: #Ignores small error specks and stuff
            continue
        x, y, bw, bh = cv2.boundingRect(cnt)
        cx = (x + bw / 2) / w
        cy = (y + bh / 2) / h
        bw_n = bw / w
        bh_n = bh / h
        #Stuff for normalization
        cnt = cnt.reshape(-1, 2)
        xs = cnt[:, 0] / w
        ys = cnt[:, 1] / h

        #class id 0 = transparent_object
        f.write(f"0 {cx:.6f} {cy:.6f} {bw_n:.6f} {bh_n:.6f}")
        for px, py in zip(xs, ys):
            f.write(f" {px:.6f} {py:.6f}")
        f.write("\n")
```

Figure 4: Code for mask conversion

Now that our masks have been converted, we needed to train the model based off of these transparent objects. Thankfully our dataset contained all kinds of transparent objects, from cups and glasses to cloches and windows, allowing the model to maximally learn the features of transparency rather than individual objects.

From here, we ran our models in an ensemble, utilizing an IoU helper to measure bounding box intersection, as shown below in figure 5.

```

def iou(box1, box2): 1 usage
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    inter = max(0, x2 - x1) * max(0, y2 - y1)
    if inter == 0:
        return 0.0

    area1 = (box1[2]-box1[0]) * (box1[3]-box1[1])
    area2 = (box2[2]-box2[0]) * (box2[3]-box2[1])
    return inter / (area1 + area2 - inter)

```

Figure 5: Code for IoU helper

From here, we match each transparent detection with the best COCO object, labeling overlapping detections and displaying them, as seen below in figures 6 and 7.

```

#Segmentation for the transparent model & coco
t_results = transparent_model(frame, conf=0.5, verbose=False)[0]
c_results = coco_model(frame, conf=0.5, verbose=False)[0]
t_boxes = t_results.bboxes # transparent detections
c_boxes = c_results.bboxes # COCO detections

annotated = t_results.plot()

#Have COCO find the best guess for the transparent object
for tb in t_boxes:
    tb_xyxy = tb.xyxy.cpu().numpy()[0] # [x1,y1,x2,y2]
    best_iou = 0.0
    best_cls_name = None

    for cb in c_boxes:
        cb_xyxy = cb.xyxy.cpu().numpy()[0]
        i = iou(tb_xyxy, cb_xyxy)
        if i > best_iou:
            best_iou = i
            cls_idx = int(cb.cls.item())
            best_cls_name = c_results.names[cls_idx]

```

```

if best_iou > 0.3 and best_cls_name is not None:
    #Draw text near the transparent box saying what COCO thinks it is
    x1, y1, x2, y2 = map(int, tb_xyxy)
    cv2.putText(
        annotated,
        text: f"transparent {best_cls_name}",
        org: (x1, max(0, y1-10)),
        cv2.FONT_HERSHEY_SIMPLEX,
        fontScale: 0.5,
        color: (255, 255, 255),
        thickness: 1,
        cv2.LINE_AA,
    )

cv2.imshow( winname: "Transparent + COCO", annotated)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```

Figures 6 & 7: Code for object detection in ensemble set-up

Experiments

When running the transparent-only model, our model was very strong at recognizing clear/transparent objects of various shapes and sizes, as can be seen below in figures 8, 9 and 10.



Figure 8: Transparency test with glass plate

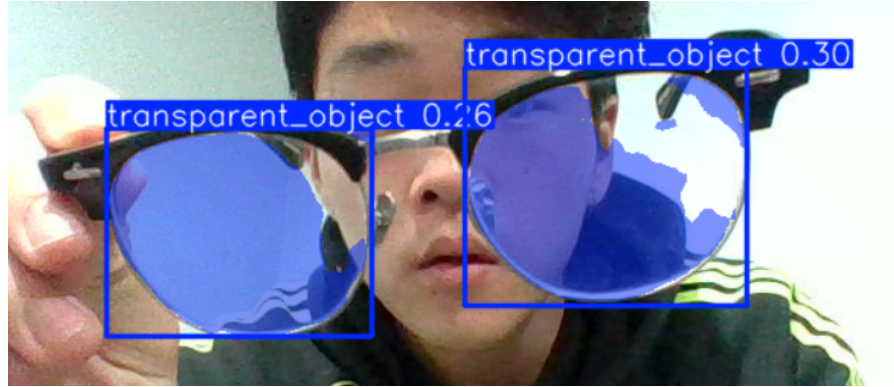


Figure 9: Transparency test with eyeglasses



Figure 10: Transparency test with cup

As can be seen above, the model was successful when predicting transparency with handheld objects, but significantly better at detecting transparency with a high degree of certainty with cups and glasses. This could most likely be from the large number of cups and glasses in the dataset utilized. To test whether the model relied overly on context, such as shape, rather than actual transparency-specific feature such as refraction and blurring, we tested it by holding a transparent pane to the camera, as seen below in figure 11.

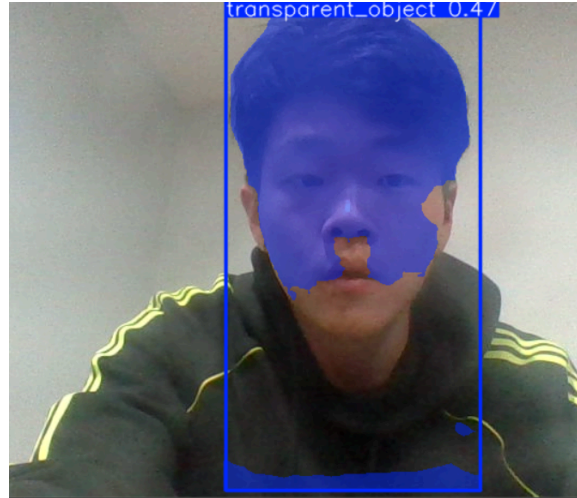


Figure 11: Viewing of object through transparent plane

While the model segmented me and built the bounding box around me as the focal point, it was still able to classify me as transparent because through the pane I met the criteria learned through training.

As a control, we ran the COCO-specific model to try and figure out how transparency affected the model's classification, an example of which can be seen below in figure 12.

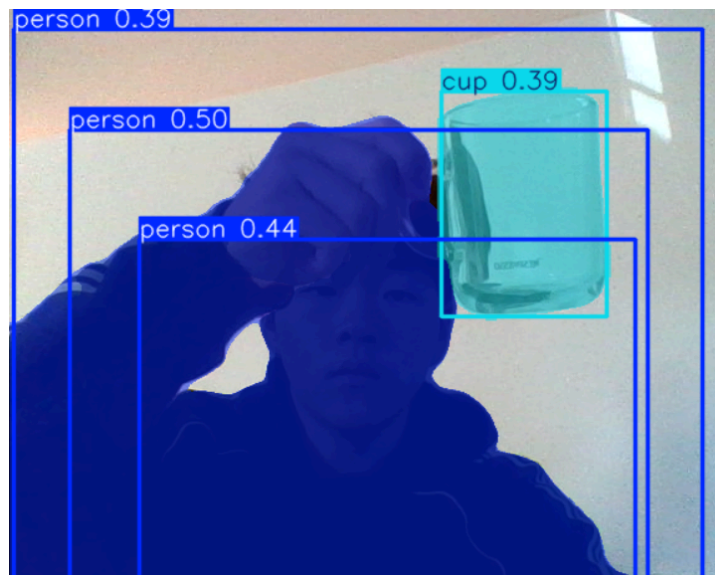


Figure 12: COCO-trained model test with a transparent object

As can be seen above, the transparent object severely impacted the model's performance, lowering the overall person score and building additional bounding boxes. This is most likely due to the

refraction of light and background images within the cup. For comparison, here is a sample using the model with no reflective images.



Figure 13: COCO-trained model test with no transparent objects

However, when running the ensemble models, we found that while there was better accuracy in bounding boxes and segmentation, the COCO-model's classification differed on transparent objects vs how they classified them regularly, as seen below in figure 14 vs figure 12.



Figure 14: Model Ensemble test

Unlike in figure 12, this model classified the cup as a wine glass, even though the base COCO model was unchanged. However, other shapes had a lot more success, such as bowls, which can be seen in figure 15.

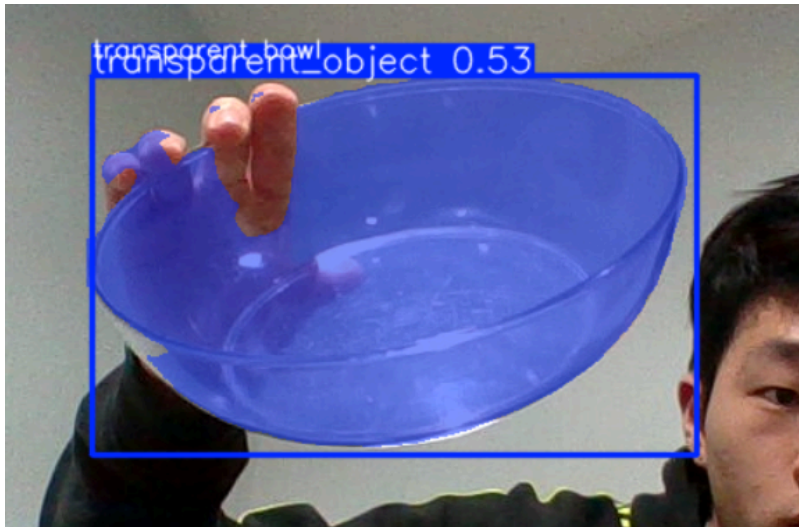


Figure 15: Models recognizing transparent bowl

Conclusion

Drones require speed, accuracy, and reliability for automation and assisted flying. In fast-paced aerial environments, with constantly shifting data, it is crucial drones are able to accurately classify obstacles to avoid crashing and route efficient flight paths. In lower-altitude applications, buildings and windows act as a huge hindrance, which is why it is important neural networks be able to recognize the key features of transparent objects. Our project was able to recognize transparency as a concept, which could serve as a base for drone obstacle-based decisions, but it was weaker when working together with wider class-based recognition models. A way to strengthen this in the future would be to fine-tune the COCO-trained model with transparent objects, creating new classes for specific variations of the object. This would require higher computing resources, which were a severe limitation for our project, but would increase reliability and certainty of the model.

References

1. Badrinarayanan, V., Kendall, A., & Cipolla, R. (2017). *SegNet: A deep convolutional encoder-decoder architecture for image segmentation*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 39(12), 2481–2495. <https://doi.org/10.1109/TPAMI.2016.2644615>
2. Schneidereit, T., Gohrenz, S., & Breuß, M. (2025). *Object detection characteristics in a learning factory environment using YOLOv8*. arXiv. <https://arxiv.org/abs/2503.10356>
3. Xie, E., Wang, W., Wang, W., Ding, M., Shen, C., & Luo, P. (2020). *Segmenting transparent objects in the wild*. arXiv. <https://arxiv.org/abs/2003.13948>